

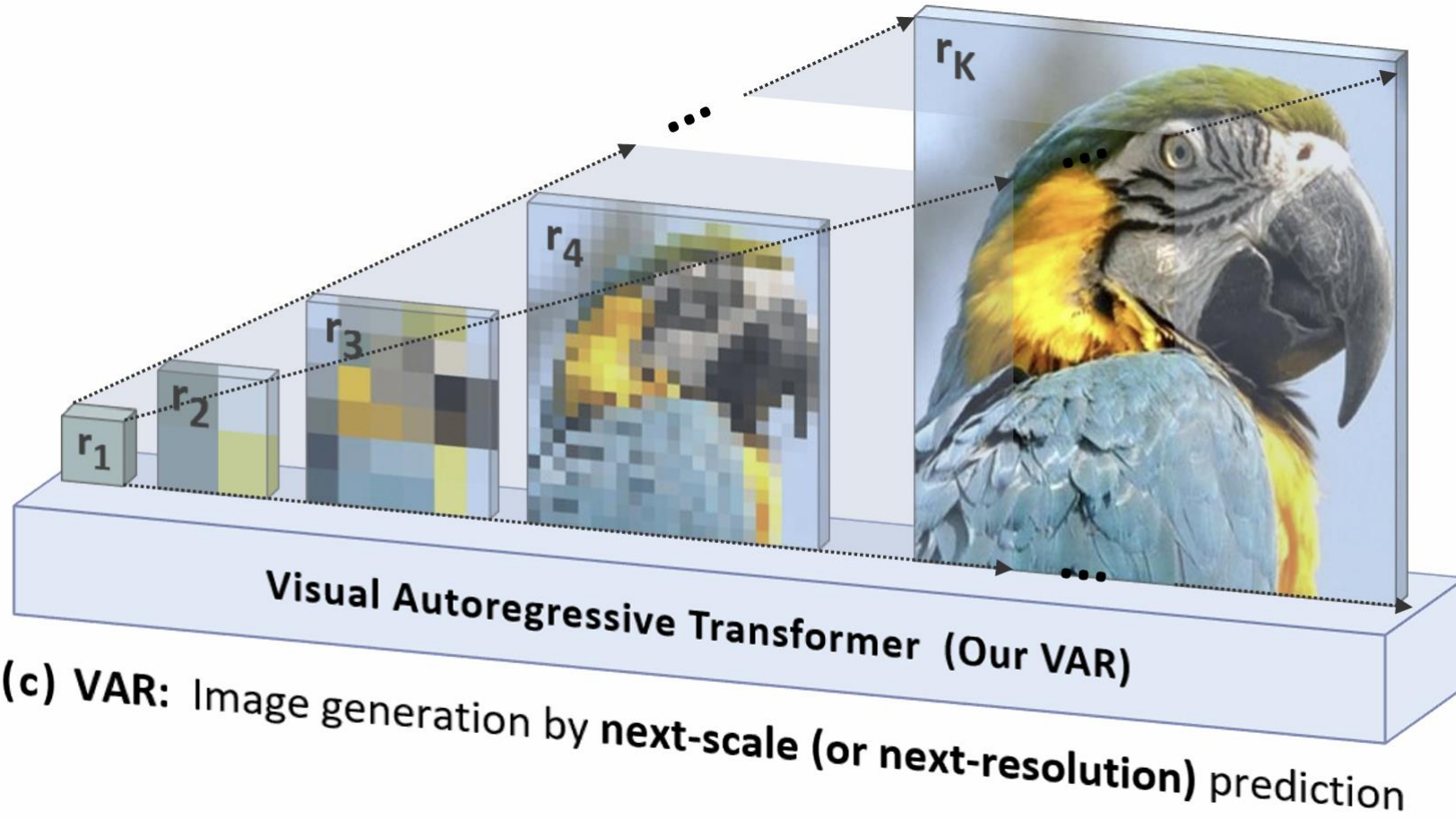
Autoregressive Modeling: Literature Review

2026/06/26

Jeongwoo Lee

leejeongwoo9941@gmail.com

Why Visual Auto Regressive Modeling?



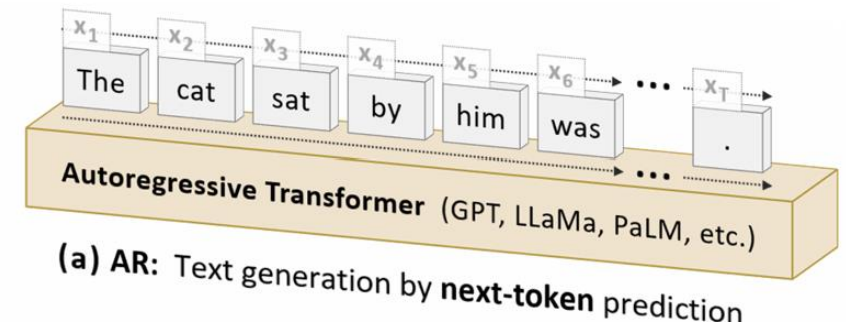
Recall) Autoregressive Modeling in Language Models

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_1, x_2, \dots, x_{t-1}).$$

$x = (x_1, x_2, \dots, x_T)$, a sequence of **discrete** tokens,
where $x_t \in [V]$, an integer from a vocabulary of size V

→ known as '**next-token prediction**'

e.g. GPT, LLaMa, ...

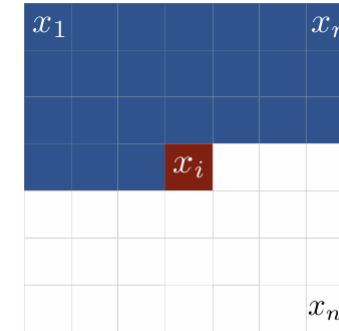


Recall) Past Autoregressive Modeling in Vision Models

PixelRNN, PixelCNN

Use pixels as discrete tokens

→ Slow generation



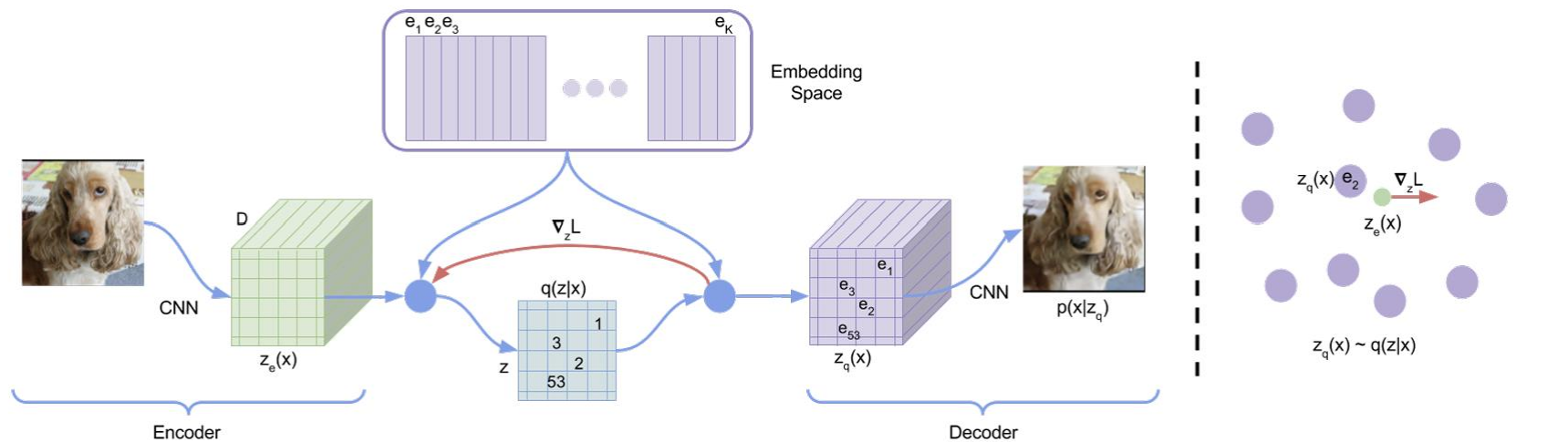
Context

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p(x_i | x_1, \dots, x_{i-1})$$

Recall) Past Autoregressive Modeling in Vision Models

How to discretize an image, instead of using pixels?

→ Convert continuous pixels into a grid of discrete visual tokens using a VQ-based tokenizer (VQVAE, VQGAN)



$f = \mathcal{E}(im),$
 im : raw image
 $\mathcal{E}$: encoder

$q^{(i,j)} = \left(\arg \min_{v \in [V]} \|\text{lookup}(Z, v) - f^{(i,j)}\|_2 \right) \in [V],$
 Quantization process

learnable codebook $Z \in \mathbb{R}^{V \times C}$

Problems of Autoregressive Modeling in Vision Models

1. Less competitive image quality than diffusion models
2. Quantization errors with a limited vocabulary size
3. Slow token-by-token generation
4. Bidirectional Correlations of token sequences $x=(x_1,...,x_{\{h \times w\}})$

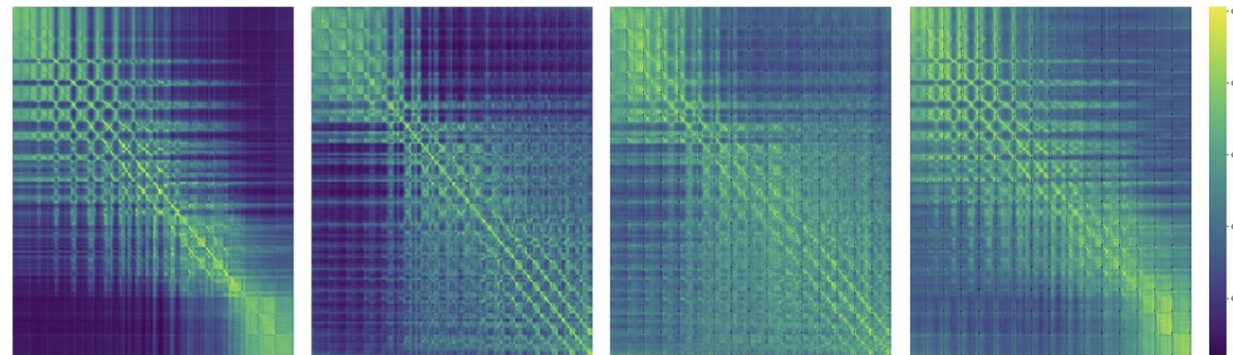
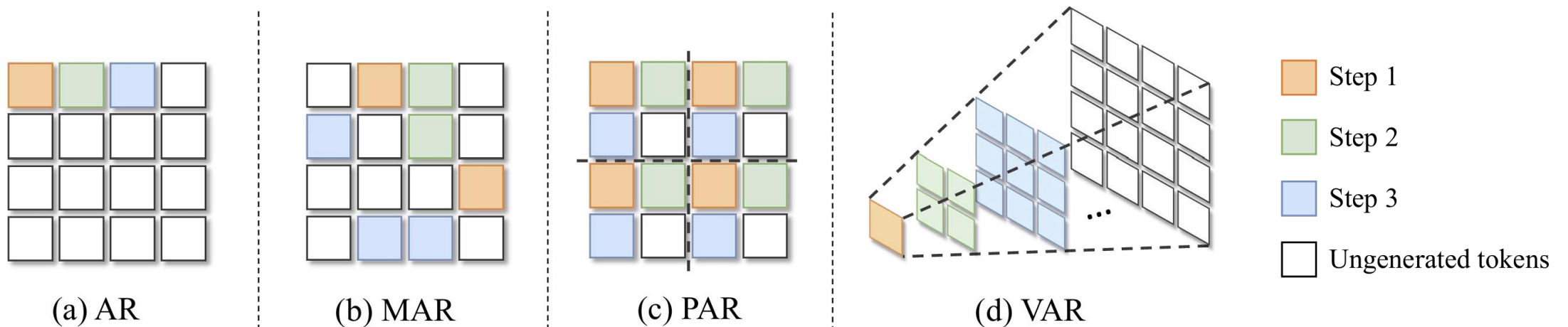


Figure 9: **Token dependency plotted.** The normalized heat map of attention scores in the last self-attention layer of VQGAN encoder is visualized. 4 random 256×256 images from ImageNet validation set are used.

Problems of Autoregressive Modeling in Vision Models

1. Less competitive image quality than diffusion models (-> VAR)
2. Quantization errors with a limited vocabulary size (-> Infinity)
3. Slow token-by-token generation (-> PAR, NAR, LPD)
4. Bidirectional Correlations of token sequences $x=(x_1,...,x_{\{h \times w\}})$ (-> VAR)



Parallelized Autoregressive Visual Generation

3. Slow token-by-token generation (-> **PAR**, NAR, LPD)

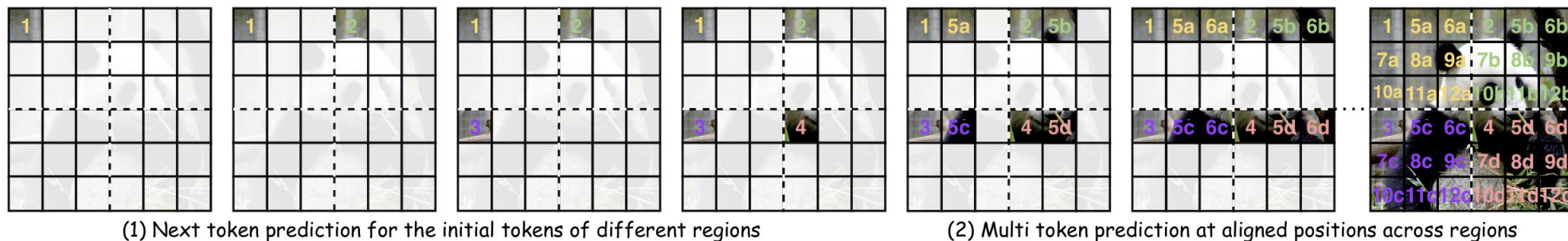


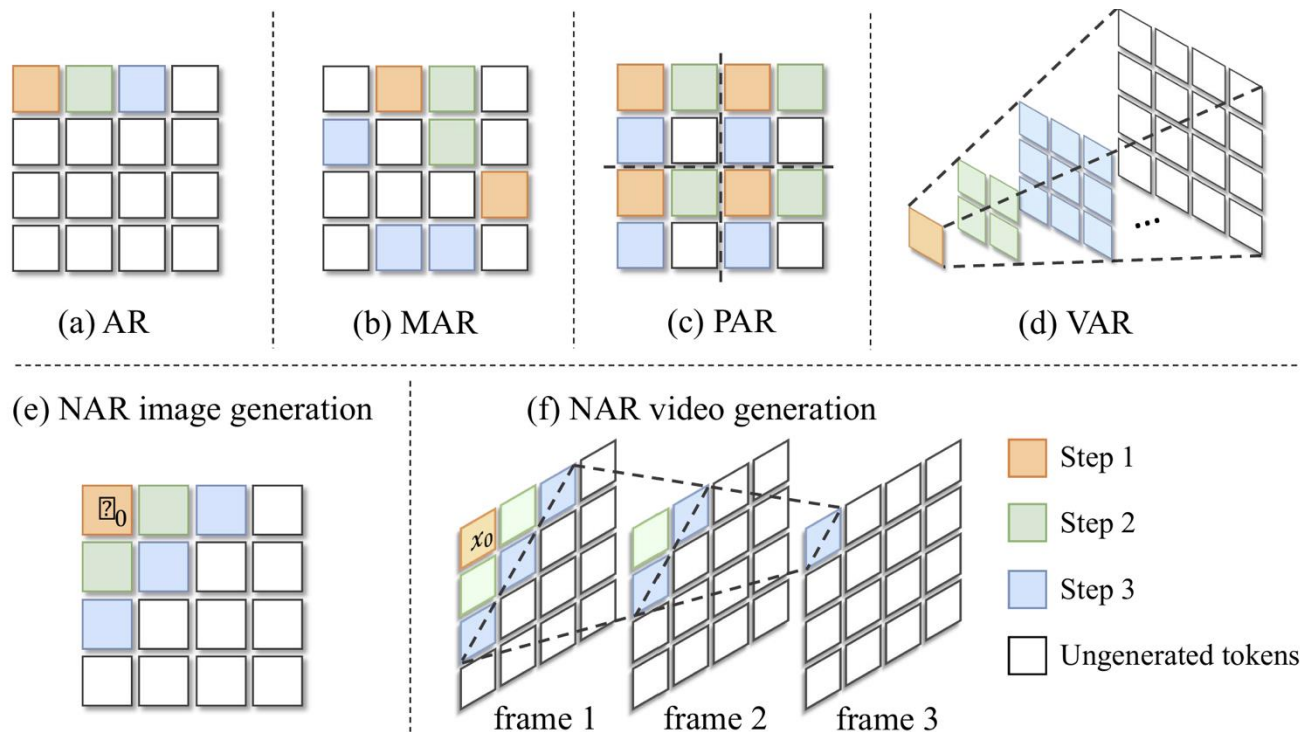
Figure 3. **Illustration of our non-local parallel generation process.** Stage 1: sequential generation of initial tokens (1-4) for each region (separated by dotted lines) to establish global structure. Stage 2: parallel generation at aligned positions across different regions (e.g., 5a-5d), then moving to next aligned positions (6a-6d, 7a-7d, etc.) for parallel generation. Same numbers indicate tokens generated in the same step, and letter suffix (a,b,c,d) denotes different regions .

PAR divide the image into **multiple regions**

and generate aligned tokens across regions **simultaneously**.

Neighboring Autoregressive Modeling for Efficient Visual Generation

3. Slow token-by-token generation (-> PAR, **NAR**, LPD)

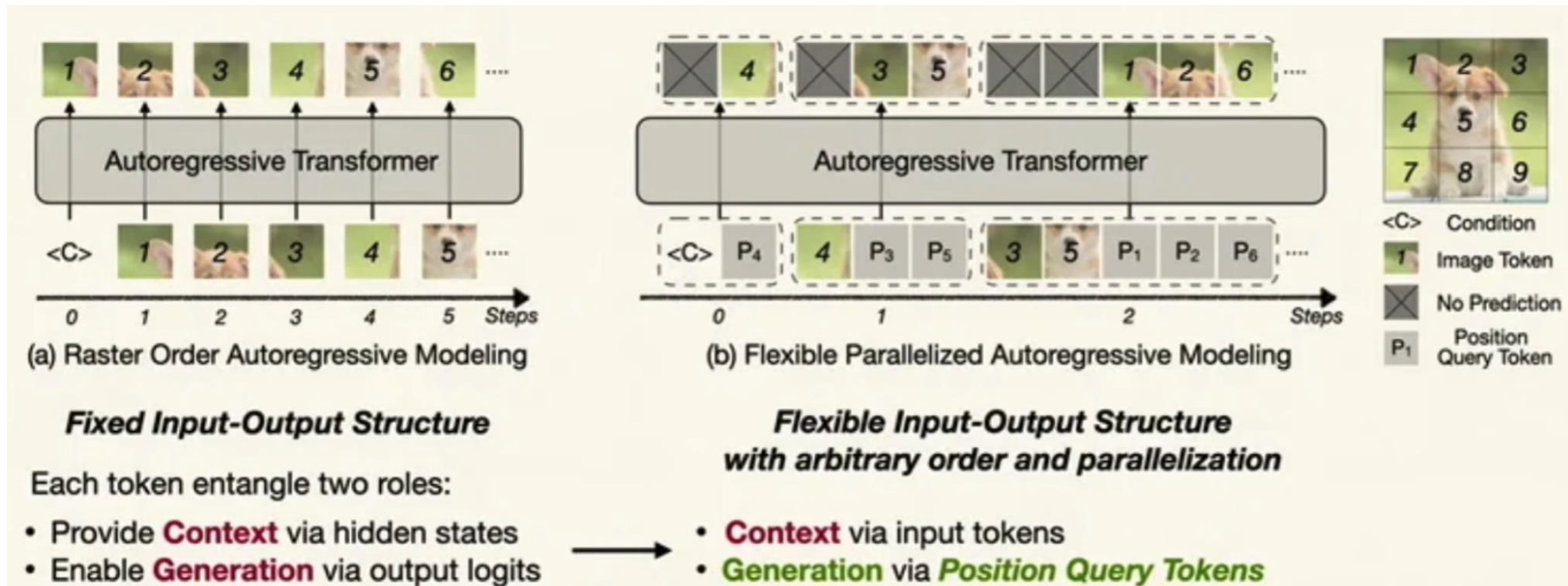


NAR starts from an initial token

and progressively expands the decoded region by predicting **neighboring** tokens in **parallel**.

Locality-aware Parallel Decoding for Efficient Autoregressive Image Generation

3. Slow token-by-token generation (-> PAR, NAR, **LPD**)



LPD moves beyond fixed raster-order decoding

by using **position query tokens** to generate **arbitrary** target positions in **parallel**.

Locality-aware Parallel Decoding for Efficient Autoregressive Image Generation

3. Slow token-by-token generation (-> PAR, NAR, **LPD**)

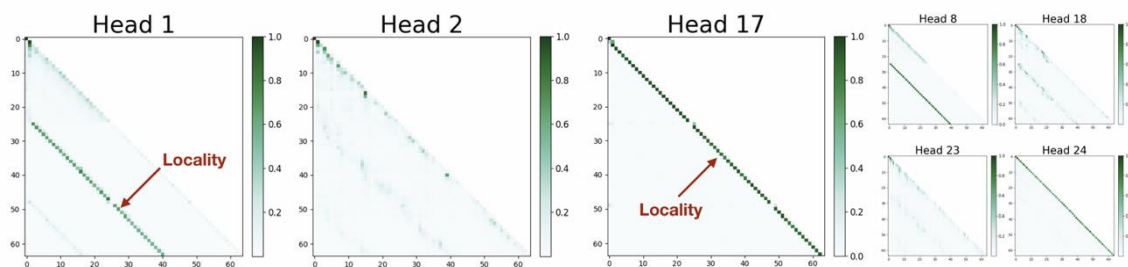
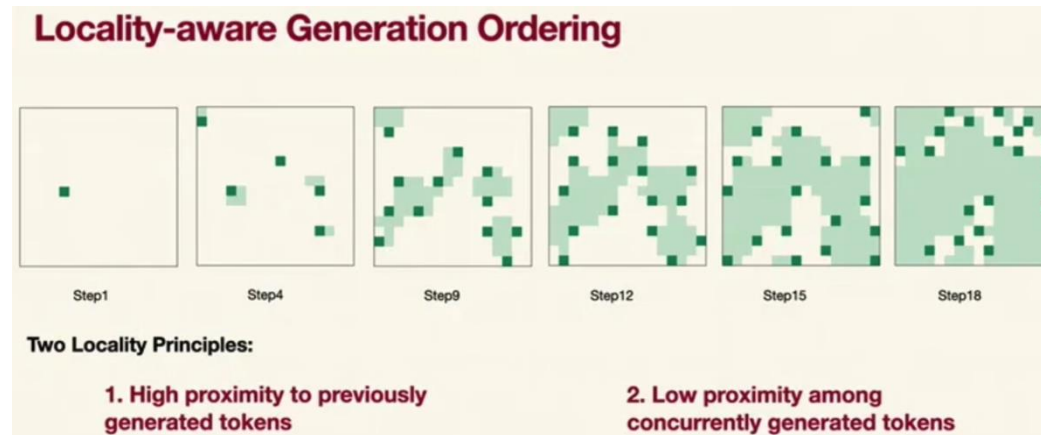


Figure 2: **Visualization of attention maps in the LLAMAGEN-1.4B model.** There is strong spatial locality, as the attention of a decoding token is concentrated on nearby spatial tokens. LLAMAGEN encodes images into 24×24 tokens, where a token that is 24 positions earlier in the attention map corresponds to the token directly above it in the 2D grid.

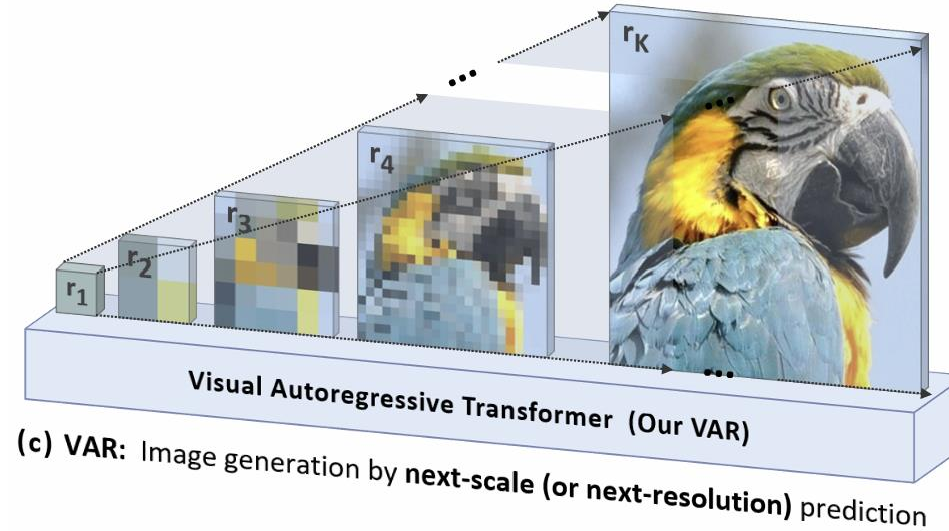


LPD uses **locality-aware generation** ordering:

generate tokens **near previously decoded context**,

but **avoid** strongly dependent **nearby tokens within the same parallel step**.

Visual AutoRegressive Model (VAR)



VAR reformulated the autoregressive modeling by **next-scale prediction**, instead of next-token prediction.

$$p(r_1, r_2, \dots, r_K) = \prod_{k=1}^K p(r_k \mid r_1, r_2, \dots, r_{k-1}),$$

$r_k \in [V]^{h_k \times w_k}$ is the token map at scale k containing $h_k \times w_k$ tokens

Result of Visual AutoRegressive Model (VAR)

Table 1: **Generative model family comparison on class-conditional ImageNet 256×256 .** “↓” or “↑” indicate lower or higher values are better. Metrics include Fréchet inception distance (FID), inception score (IS), precision (Pre) and recall (rec). “#Step”: the number of model runs needed to generate an image. Wall-clock inference time relative to VAR is reported. Models with the suffix “-re” used rejection sampling. †: taken from MaskGIT [17].

Type	Model	FID↓	IS↑	Pre↑	Rec↑	#Para	#Step	Time
GAN	BigGAN [13]	6.95	224.5	0.89	0.38	112M	1	—
GAN	GigaGAN [42]	3.45	225.5	0.84	0.61	569M	1	—
GAN	StyleGan-XL [74]	2.30	265.1	0.78	0.53	166M	1	0.3 [74]
Diff.	ADM [26]	10.94	101.0	0.69	0.63	554M	250	168 [74]
Diff.	CDM [36]	4.88	158.7	—	—	—	8100	—
Diff.	LDM-4-G [70]	3.60	247.7	—	—	400M	250	—
Diff.	DiT-L/2 [63]	5.02	167.2	0.75	0.57	458M	250	31
Diff.	DiT-XL/2 [63]	2.27	278.2	0.83	0.57	675M	250	45
Diff.	L-DiT-3B [3]	2.10	304.4	0.82	0.60	3.0B	250	>45
Diff.	L-DiT-7B [3]	2.28	316.2	0.83	0.58	7.0B	250	>45
Mask.	MaskGIT [17]	6.18	182.1	0.80	0.51	227M	8	0.5 [17]
Mask.	RCG (cond.) [51]	3.49	215.5	—	—	502M	20	1.9 [51]
AR	VQVAE-2† [68]	31.11	~45	0.36	0.57	13.5B	5120	—
AR	VQGAN† [30]	18.65	80.4	0.78	0.26	227M	256	19 [17]
AR	VQGAN [30]	15.78	74.3	—	—	1.4B	256	24
AR	VQGAN-re [30]	5.20	280.3	—	—	1.4B	256	24
AR	ViTVQ [92]	4.17	175.1	—	—	1.7B	1024	>24
AR	ViTVQ-re [92]	3.04	227.4	—	—	1.7B	1024	>24
AR	RQTran. [50]	7.55	134.0	—	—	3.8B	68	21
AR	RQTran.-re [50]	3.80	323.7	—	—	3.8B	68	21
VAR	VAR-d16	3.30	274.4	0.84	0.51	310M	10	0.4
VAR	VAR-d20	2.57	302.6	0.83	0.56	600M	10	0.5
VAR	VAR-d24	2.09	312.9	0.82	0.59	1.0B	10	0.6
VAR	VAR-d30	1.92	323.1	0.82	0.59	2.0B	10	1
VAR	VAR-d30-re	1.73	350.2	0.82	0.60	2.0B	10	1
	(validation data)	1.78	236.9	0.75	0.67			

VAR has **faster inference time** than diffusion models and previous AR models.

Also, VAR is comparable or even **outperforms** diffusion models in class-conditional generation.

Problems of Visual AutoRegressive Model (VAR)

1. Quantization bottleneck from **finite VQ codebook** (VAR also uses a VQVAE) (-> HART, Infinity)
2. Computation & Memory Overhead from multi-scale token modeling

Poor reconstruction performance of discrete tokenizer



Ground Truth



Existing VQVAE Reconstruction

limited vocabulary !

obvious distortion !

HART: Efficient Visual Generation with Hybrid Autoregressive Transformer

1. Quantization bottleneck from **finite VQ codebook** (VAR also uses a VQVAE) (-> **HART**, Infinity)

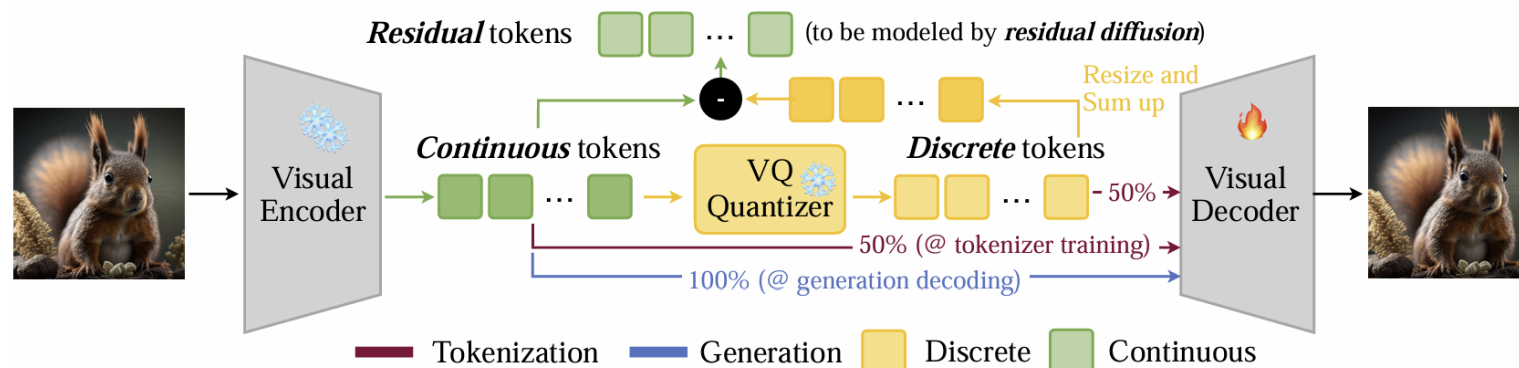


Figure 5: Unlike conventional image tokenizers that decode either continuous *or* discrete latents, the **hybrid tokenizer** in HART is trained to decode both continuous *and* discrete tokens. At inference time, we only decode continuous tokens, which are the sum of discrete tokens and residual tokens. The residual tokens will be modeled by residual diffusion (introduced in Figure 6).

HART **combines discrete** tokens for global structure and **continuous residual** tokens for fine details, reducing the reconstruction loss of VQ-based tokenization.

Infinity: Scaling Bitwise AutoRegressive Modeling for High-Resolution Image Synthesis

1. Quantization bottleneck from **finite VQ codebook** (VAR also uses a VQVAE) (-> HART, **Infinity**)

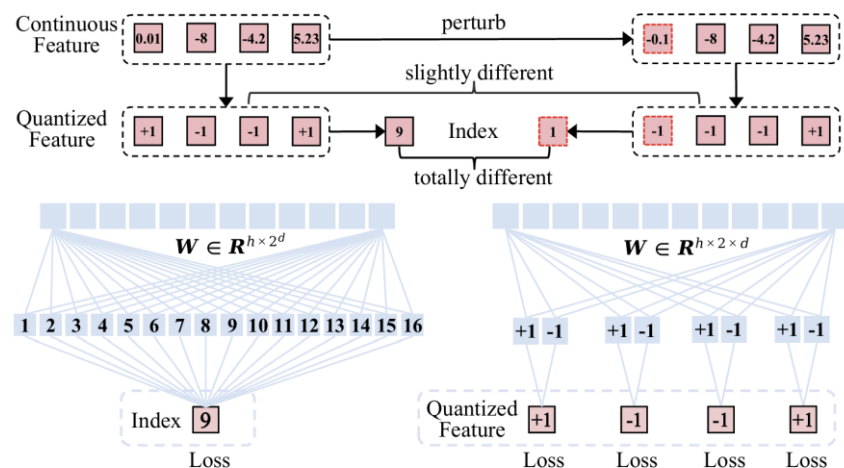
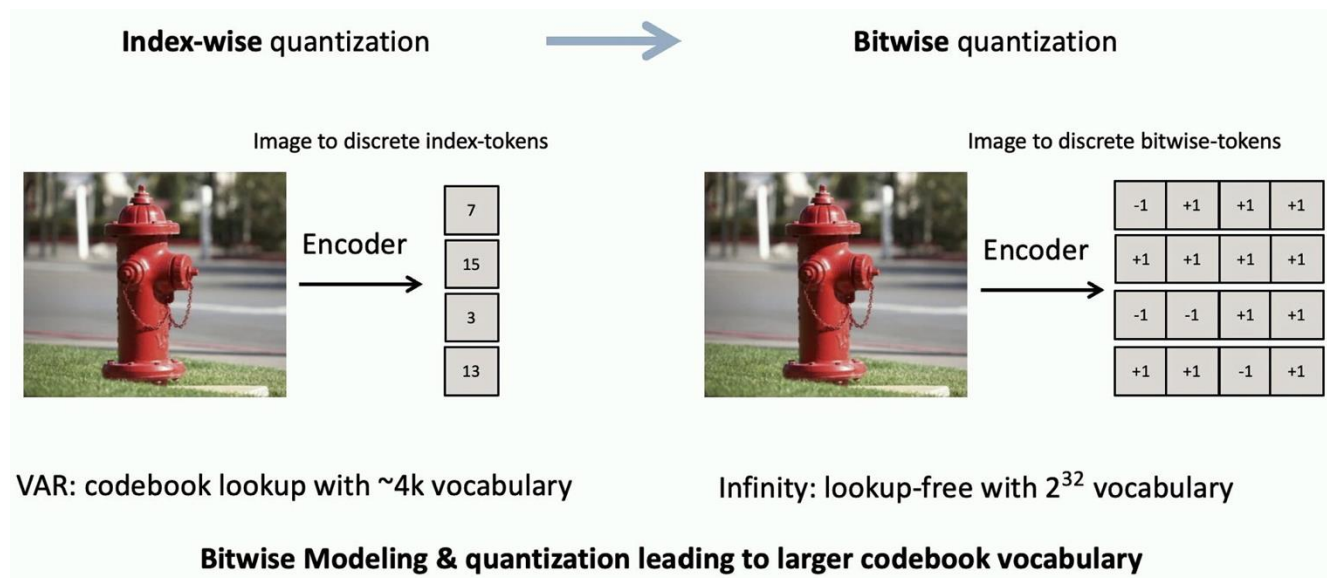


Figure 2: Visual tokenizer quantizes continuous features and then gets index labels. Conventional classifier (left) predicts 2^d indices. Infinite-Vocabulary Classifier (right) predicts d bits instead. Slight perturbations to near-zero values in continuous features cause a complete change of index labels. Bit labels (*i.e.* quantized features) change subtly and still provide steady supervision. Besides, parameters of conventional classifiers grow exponentially as d increases, while IVC grows linearly. If $d = 32$ and $h = 2048$, the conventional classifier requires **8.8 trillion** parameters, exceeding current compute limits. By contrast, IVC only requires **0.13M** parameters.



Infinity replaces index-wise VQ token prediction with **bitwise prediction**, enabling an extremely **large vocabulary** while reducing quantization errors.

Problems of Visual AutoRegressive Model (VAR)

1. Quantization bottleneck from finite VQ codebook (VAR also uses a VQVAE) (-> HART, Infinity)
2. **Computation & Memory Overhead** from multi-scale token modeling (-> **training-free token reduction**)

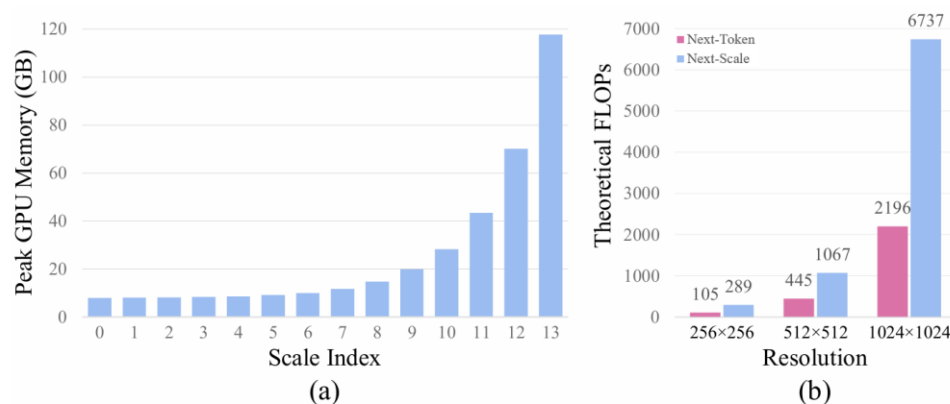


Figure 2. (a) Peak GPU memory consumption across different scales during 1024×1024 image generation in VAR. As the scale increases, the token sequence length grows rapidly, leading to quadratic growth in attention computation complexity. (b) Theoretical GFLOPs required by next-token and next-scale prediction models to generate images of various resolutions under uniform settings. Compared to the raster-scan next-token prediction, VAR’s coarse-to-fine next-scale prediction additionally requires predicting feature maps at multiple previous scales before generating the final scale. The cumulative computation at these previous transition scales introduces extra computational cost to VAR.

Problems of Visual AutoRegressive Model (VAR)

1. Computation & Memory Overhead from multi-scale token modeling (-> training-free token reduction)

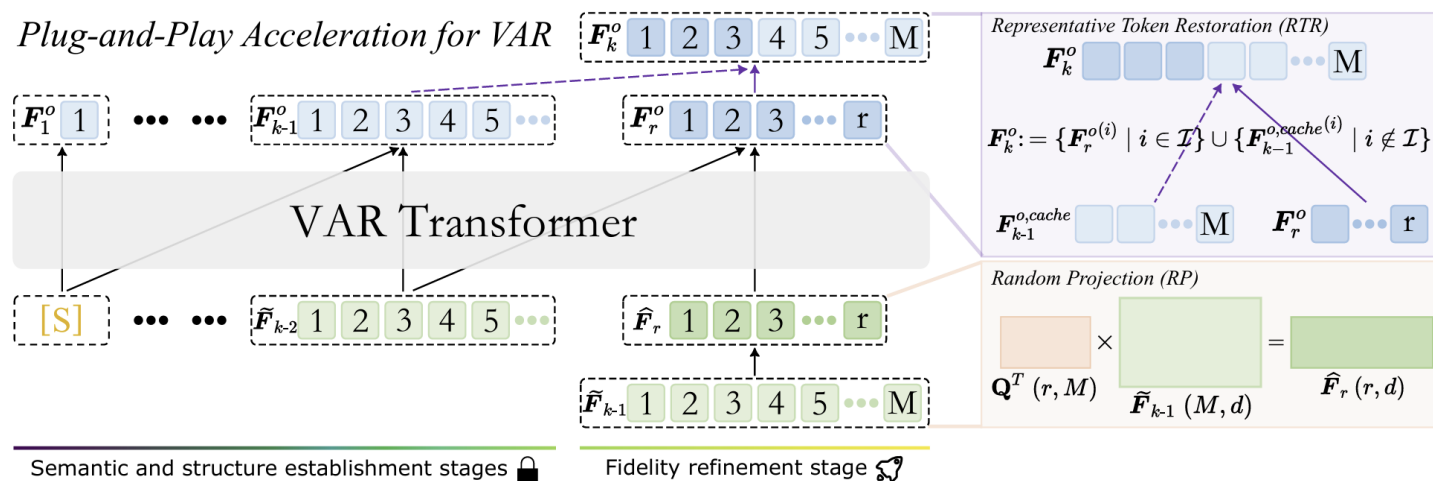


Figure 4. Overview of the proposed **FasterVAR** framework. We retain the original VAR inference process for the semantic and structure establishment stages, while exploiting semantic irrelevance and low-rank properties in the fidelity refinement stage to accelerate inference.

Table 4. Quantitative comparison of FID, KID, and IS on COCO2014 and COCO2017.

Methods	#Speed↑	COCO2014-30K			COCO2017-5K		
		FID↓	KID×10 ² ↓	IS↑	FID↓	KID×10 ² ↓	IS↑
Infinity	1.0×	26.64	1.26	42.61	35.82	1.31	37.21
Ours	3.4×	26.91	1.36	42.18	37.13	1.42	37.70